Programarea calculatoarelor

FMI

Secția Calculatoare și tehnologia informației, anul I

Cursul 3 / 20.10.2025

Programa cursului

1. Introducere

- Algoritmi
- Limbaje de programare.
- Introducere în limbajul C. Structura unui program C.

2. Fundamentele limbajului C

- Etapele realizării unui program C.
- Tipuri de date fundamentale. Variabile. Constante. Operatori. Expresii. Conversii.
- Tipuri derivate de date: tablouri, șiruri de caractere, structuri, uniuni, câmpuri de biți, enumerări, pointeri
- Instrucțiuni de control
- Directive de preprocesare. Macrodefiniții.
- Funcții de citire/scriere.

3. Fișiere text

- Funcții specifice de manipulare.

4. Funcții (1)

- Declarare și definire. Apel. Metode de trasmitere a paramerilor. Pointeri la funcții.

5. Tablouri și pointeri

- Legătura dintre tablouri și pointeri
- Aritmetica pointerilor
- Alocarea dinamică a memoriei
- Clase de memorare

6. Șiruri de caractere

- Funcții specifice de manipulare

7. Fișiere binare

- Funcții specifice de manipulare

8. Structuri de date complexe și autoreferite

- Definire și utilizare

9. Funcții (2)

- Funcții cu număr variabil de argumente
- Preluarea argumentelor funcției main din linia de comandă
- Programare generică

Cursul de azi

1. **Pointeri**
2. **Tablouri**
3. **Șiruri de caractere**
4. **Structuri**

Pointeri

- pointer = tip de date derivat folosit pentru manipularea adreselor de memorie
- **sintaxa:**

tip *nume_variabilă;

tip = tipul de bază al variabilei de tip pointer nume_variabilă

***** = **operator de indirectare**

nume_variabila = variabila de tip pointer care poate lua ca valori adrese de memorie

- **cel mai puternic mecanism de accesare a memoriei în C**

Pointeri

- pointer = tip de date derivat folosit pentru manipularea adreselor de memorie
- operatori pentru manipularea adreselor de memorie:

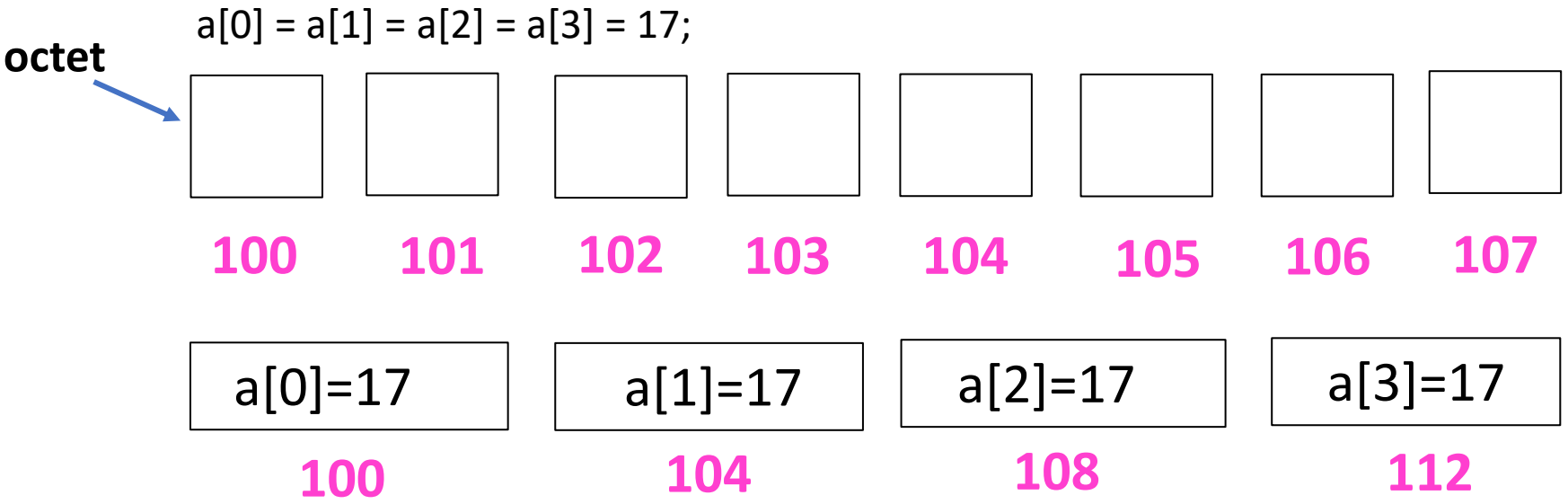
  & - operatorul de referențiere

  &variabila – furnizează adresa variabilei respective

  *** - operatorul de dereferențiere**

  *variabila_de_tip_pointer – furnizează valoarea aflată la adresa de memorie stocată în variabila_de_tip_pointer

Memorie



În principiu:

- fiecare locație de memorie are adresa unica
- fiecare locație de memorie conține o valoare

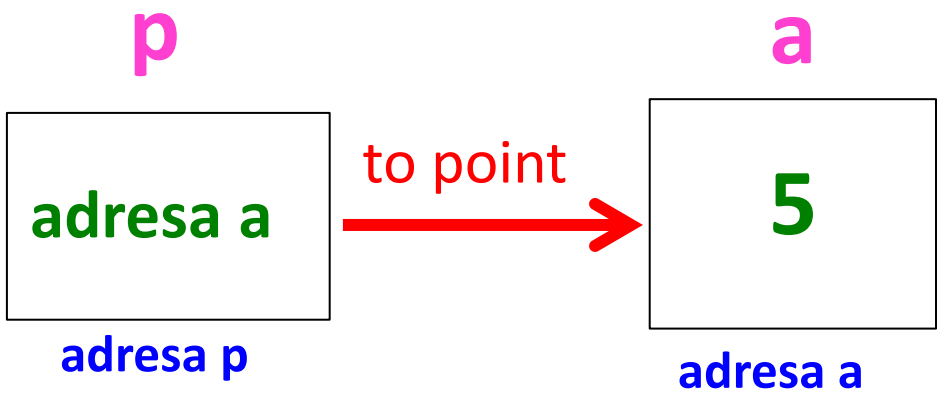
Nu putem lucra direct cu adrese de memorie! =>

=> De aceea avem nevoie de **variabile de tip adresă**!

Pointeri

Exemplu:

```
int a=5;
int *p;
p = &a;
```




```c
#include <stdio.h>

int main()
{
    int a = 5, *p;
    p = &a;
    printf("Adresa lui a: %d \n", &a);
    printf("Valoarea stocata la adresa lui a: %d \n\n", a);

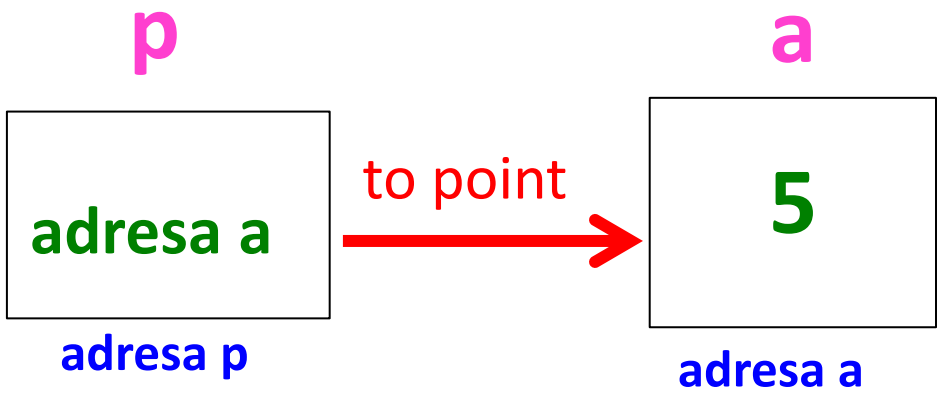
    printf("Adresa lui p: %d \n", &p);
    printf("Valoarea stocata la adresa lui p: %d \n", p);
    printf("Valoarea variabilei a carei adresa e stocata in p: %d \n", *p);

    return 0;
}
```

Pointeri

Exemplu:

```
int a=5;
int *p;
p = &a;
```




```c
#include <stdio.h>

int main()
{
    int a = 5, *p;
    p = &a;
    printf("Adresa lui a: %d \n", &a);
    printf("Valoarea stocata la adresa lui a: %d \n\n", a);

    printf("Adresa lui p: %d \n", &p);
    printf("Valoarea stocata la adresa lui p: %d \n", p);
    printf("Valoarea variabilei a carei adresa e stocata in p: %d \n", *p);

    return 0;
}
```

```
Adresa lui a: 1278772828
Valoarea stocata la adresa lui a: 5

Adresa lui p: 1278772816
Valoarea stocata la adresa lui p: 1278772828
Valoarea variabilei a carei adresa e stocata in p: 5
```

Pointeri

Exemplu: **modificarea valorii unei variabile prin dereferențiere**

```c
#include <stdio.h>

int main()
{
    int a = 5, *p;
    p = &a;

    printf("Valoarea stocata la adresa lui a: %d \n", a);

    a += 10; //acces direct
    printf("Valoarea stocata la adresa lui a: %d \n", a);

    *p += 20; //acces indirect
    printf("Valoarea stocata la adresa lui a: %d \n", a);

    return 0;
}
```

Pointeri

Exemplu: **modificarea valorii unei variabile prin dereferențiere**

```
Valoarea stocata la adresa lui a: 5
Valoarea stocata la adresa lui a: 15
Valoarea stocata la adresa lui a: 35
```

```c
#include <stdio.h>

int main()
{
    int a = 5, *p;
    p = &a;

    printf("Valoarea stocata la adresa lui a: %d \n", a);

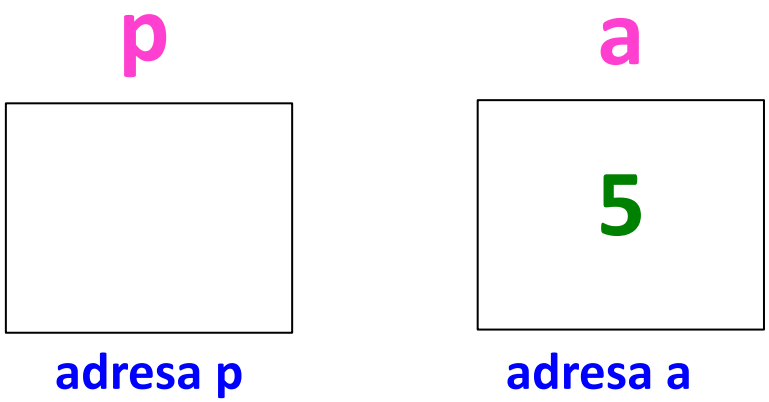
    a += 10; //acces direct
    printf("Valoarea stocata la adresa lui a: %d \n", a);

    *p += 20; //acces indirect
    printf("Valoarea stocata la adresa lui a: %d \n", a);

    return 0;
}
```

Pointeri

Exemplu: **pointeri neinițializați**




```c
#include <stdio.h>

int main()
{
    int a = 5, *p;
    //p = &a;

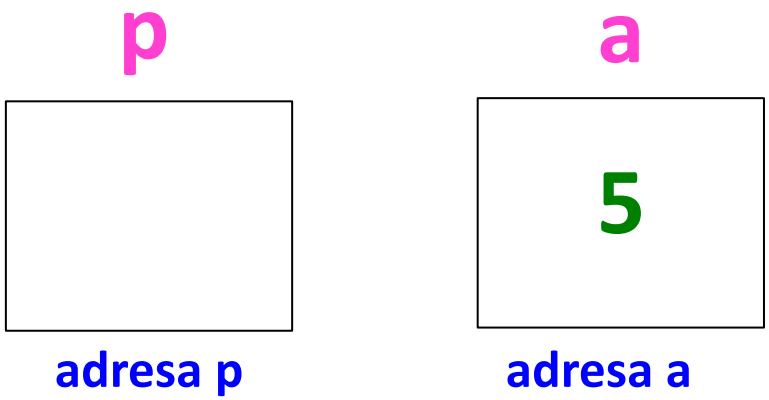
    printf("a = %d \n", a);

    *p = 10;
    printf("a = %d \n", a);

    return 0;
}
```

Pointeri

Exemplu: **pointeri neinițializați**




```c
#include <stdio.h>

int main()
{
    int a = 5, *p;
    //p = &a;

    printf("a = %d \n", a);

    *p = 10;
    printf("a = %d \n", a);

    return 0;
}
```

```
a = 5
a = 5
```

Pointeri

Exemplu: **spațiul de memorie ocupat de o variabilă de tip pointer**

```c
#include <stdio.h>

int main()
{
    int a = 5, *p;
    p = &a;

    printf("Variabila a ocupa %d octeti. \n", sizeof(a));
    printf("Variabila de tip pointer p ocupa %d octeti. \n", sizeof(p));

    return 0;
}
```

Pointeri

Exemplu: **spațiul de memorie ocupat de o variabilă de tip pointer**

```c
#include <stdio.h>

int main()
{
    int a = 5, *p;
    p = &a;

    printf("Variabila a ocupa %d octeti. \n", sizeof(a));
    printf("Variabila de tip pointer p ocupa %d octeti. \n", sizeof(p));

    return 0;
}
```

```
Variabila a ocupa 4 octeti.
Variabila de tip pointer p ocupa 8 octeti.
```

Pointeri

- **adrese de memorie**
  - ✓ în C există un specificator de format special **(%p)** pentru tipărirea valorilor reprezentând adresele de memorie.

```c
#include <stdio.h>

int main()
{
    int x;

    printf("Adresa lui x este %p \n", &x);
    printf("Adresa lui x este %x \n", &x);
    printf("Adresa lui x este %d \n", &x);

    return 0;
}
```

Pointeri

- **adrese de memorie**
  - ✓ în C există un specificator de format special **(%p)** pentru tipărirea valorilor reprezentând adresele de memorie.

```c
#include <stdio.h>

int main()
{
    int x;

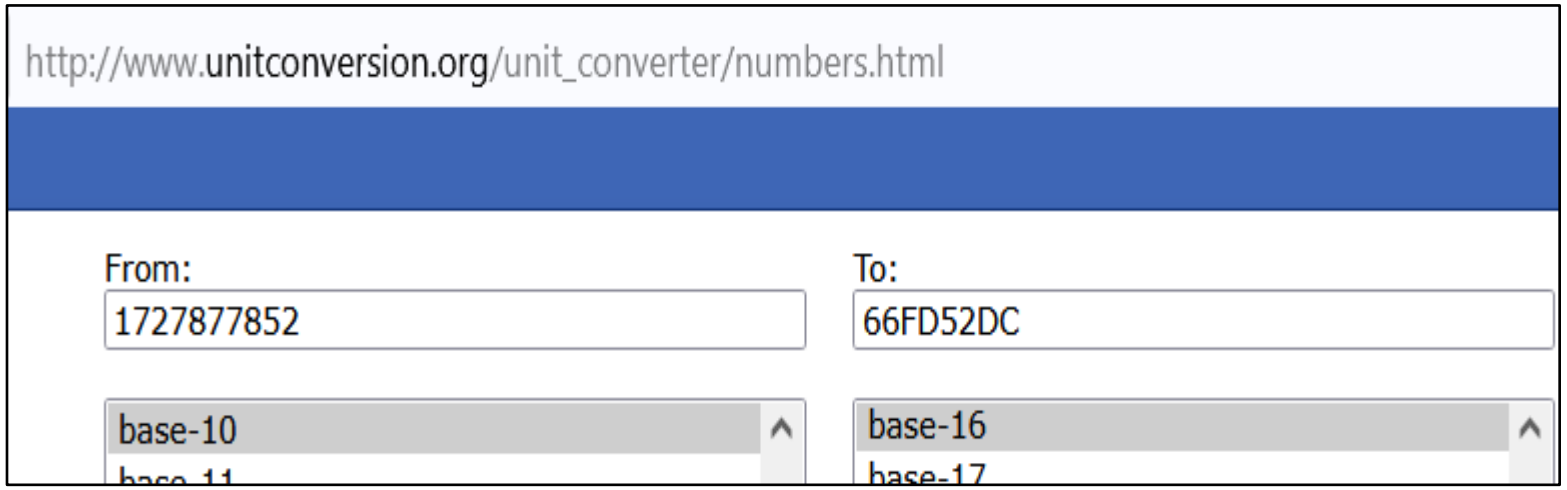
    printf("Adresa lui x este %p \n", &x);
    printf("Adresa lui x este %x \n", &x);
    printf("Adresa lui x este %d \n", &x);

    return 0;
}
```

```
Adresa lui x este 0x7ffc66fd52dc
Adresa lui x este 66fd52dc
Adresa lui x este 1727877852
```

Pointeri

- **adrese de memorie**
  - ✓ în C există un specificator de format special **(%p)** pentru tipărirea valorilor reprezentând adresele de memorie.



Verificarea valorilor obținute la o rulare a codului din exemplul anterior.

Cursul de azi

1. Pointeri
2. **Tablouri**
3. Șiruri de caractere
4. Structuri

Tablouri unidimensionale (vectori)

Tablouri bidimensionale (matrice)

Tablouri unidimensionale

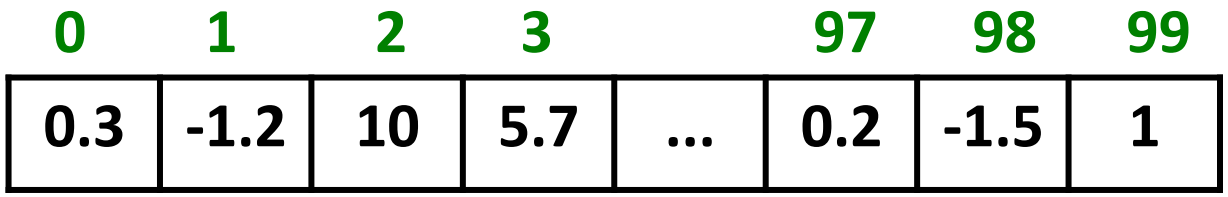
- **sintaxa:**

  tip nume_tablou [dimensiune];

- *exemple:*

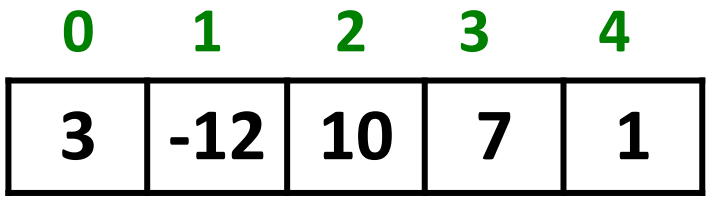
double v[100];

v[3] = 5.7;



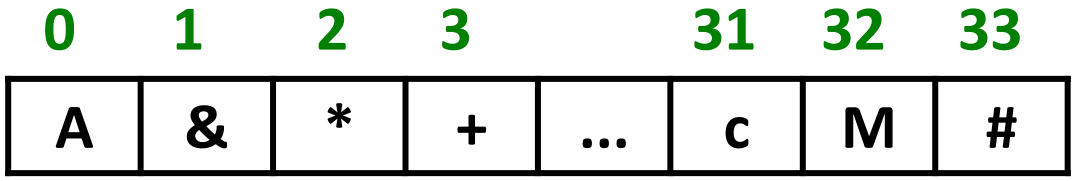
int a[5];

a[0] =3;



char c[34];

c[1] ='&';



- **în C, primul element al unui tablou are indicele 0.**

Tablouri unidimensionale

Definiție: **set de valori de același tip memorat la adrese succesive de memorie**

```c
#include <stdio.h>

int main()
{
    int a[5], i;
    for(i = 0; i < 4; i++)
        printf("Adresa elementului %d din tabloul a: %p \n", i, &a[i]);
    printf("********************************************** \n");

    double v[100];
    for(i = 0; i < 4; i++)
        printf("Adresa elementului %d din tabloul v: %p \n", i, &v[i]);
    printf("********************************************** \n");

    char c[34];
    for(i = 0; i < 4; i++)
        printf("Adresa elementului %d din tabloul c: %p \n", i, &c[i]);
    printf("********************************************** \n");

    return 0;
}
```

Tablouri unidimensionale

Definiție: **set de valori de același tip memorat la adrese succesive de memorie**

```
Adresa elementului 0 din tabloul a: 0x7ffc851048d0
Adresa elementului 1 din tabloul a: 0x7ffc851048d4
Adresa elementului 2 din tabloul a: 0x7ffc851048d8
Adresa elementului 3 din tabloul a: 0x7ffc851048dc
****************************************
Adresa elementului 0 din tabloul v: 0x7ffc851045b0
Adresa elementului 1 din tabloul v: 0x7ffc851045b8
Adresa elementului 2 din tabloul v: 0x7ffc851045c0
Adresa elementului 3 din tabloul v: 0x7ffc851045c8
****************************************
Adresa elementului 0 din tabloul c: 0x7ffc85104580
Adresa elementului 1 din tabloul c: 0x7ffc85104581
Adresa elementului 2 din tabloul c: 0x7ffc85104582
Adresa elementului 3 din tabloul c: 0x7ffc85104583
****************************************
```



Tablouri unidimensionale

- **memorie:**
  - ✓ cantitatea de memorie necesară pentru stocarea unui tablou este direct proporțională cu tipul de date și mărimea sa.

tip nume [dimensiune] → **sizeof(nume) = sizeof (tip) * dimensiune ;**

```c
#include <stdio.h>

int main()
{
    int a[5];
    double v[100];
    char c[34];

    printf("Stocarea tabloului a necesita %d octeti. \n", sizeof(a));
    printf("Stocarea tabloului v necesita %d octeti. \n", sizeof(v));
    printf("Stocarea tabloului c necesita %d octeti. \n", sizeof(c));

    return 0;
}
```

Tablouri unidimensionale

- **memorie:**
  - ✓ cantitatea de memorie necesară pentru stocarea unui tablou este direct proporțională cu tipul de date și mărimea sa.

tip nume [dimensiune] → **sizeof(nume) = sizeof (tip) * dimensiune ;**

```
Stocarea tabloului a necesita 20 octeti.
Stocarea tabloului v necesita 800 octeti.
Stocarea tabloului c necesita 34 octeti.
```

```c
#include <stdio.h>

int main()
{
    int a[5];
    double v[100];
    char c[34];

    printf("Stocarea tabloului a necesita %d octeti. \n", sizeof(a));
    printf("Stocarea tabloului v necesita %d octeti. \n", sizeof(v));
    printf("Stocarea tabloului c necesita %d octeti. \n", sizeof(c));

    return 0;
}
```

Tablouri unidimensionale

Exemplu de inițializare

Ce afișează programul?

```c
#include <stdio.h>

int main()
{
    int a[4], x=100;
    printf("x=%d\n",x);
    a[0] = a[1] = a[2] = a[3] = 17;

    printf("x=%d\n",a[2]);

    return 0;
}
```

Tablouri unidimensionale

Exemplu de inițializare

```
1  #include <stdio.h>
2
3  int main()
4  {
5      int a[4], x=100;
6      printf("x=%d\n",x);
7      a[0] = a[1] = a[2] = a[3] = 17;
8
9      printf("x=%d\n",a[2]);
10
11     return 0;
12 }
```

x=100

x=17

Tablouri unidimensionale

Exemplu de inițializare

Ce afișează programul?

```c
#include <stdio.h>

int main()
{
    int a[4], x=100;
    printf("x=%d\n",x);
    a[0] = a[1] = a[2] = a[3] = 17;

    printf("x=%d\n",a[2]);
    printf("Adresa lui a[2] este %p \n",&a[2]);
    printf("Adresa lui x este %p \n",&x);
    return 0;
}
```

Tablouri unidimensionale

Exemplu de inițializare

```
x=100
x=17
Adresa lui a[2] este 0x7ffc3dabaad8
Adresa lui x este 0x7ffc3dabaacc
```

```c
#include <stdio.h>

int main()
{
    int a[4], x=100;
    printf("x=%d\n",x);
    a[0] = a[1] = a[2] = a[3] = 17;

    printf("x=%d\n",a[2]);
    printf("Adresa lui a[2] este %p \n",&a[2]);
    printf("Adresa lui x este %p \n",&x);
    return 0;
}
```

Tablouri bidimensionale

- **sintaxa**

tip nume_variabila [dimensiune1][dimensiune2];

Exemplu

int a[3][5];

a[1][4] = 41;

	0	1	2	3	4
0	3	-12	10	7	1
1	10	2	0	-7	41
2	-3	-2	0	0	2

Tablouri bidimensionale

Definiție: **set de valori de același tip memorat la adrese succesive de memorie**

```c
#include <stdio.h>

int main()
{
    int i, j;
    int a[3][5];

    for(i = 0; i < 3; i++)
      for(j = 0; j < 5; j++)
        printf("Adresa elementului a[%d][%d] este %p. \n", i, j, &a[i][j]);

    return 0;
}
```

Tablouri bidimensionale

Definiție: **set de valori de același tip memorat la adrese succesive de memorie**

```
Adresa elementului a[0][0] este 0x7fff34dd6a10.
Adresa elementului a[0][1] este 0x7fff34dd6a14.
Adresa elementului a[0][2] este 0x7fff34dd6a18.
Adresa elementului a[0][3] este 0x7fff34dd6a1c.
Adresa elementului a[0][4] este 0x7fff34dd6a20.
Adresa elementului a[1][0] este 0x7fff34dd6a24.
Adresa elementului a[1][1] este 0x7fff34dd6a28.
Adresa elementului a[1][2] este 0x7fff34dd6a2c.
Adresa elementului a[1][3] este 0x7fff34dd6a30.
Adresa elementului a[1][4] este 0x7fff34dd6a34.
Adresa elementului a[2][0] este 0x7fff34dd6a38.
Adresa elementului a[2][1] este 0x7fff34dd6a3c.
Adresa elementului a[2][2] este 0x7fff34dd6a40.
Adresa elementului a[2][3] este 0x7fff34dd6a44.
Adresa elementului a[2][4] este 0x7fff34dd6a48.
```

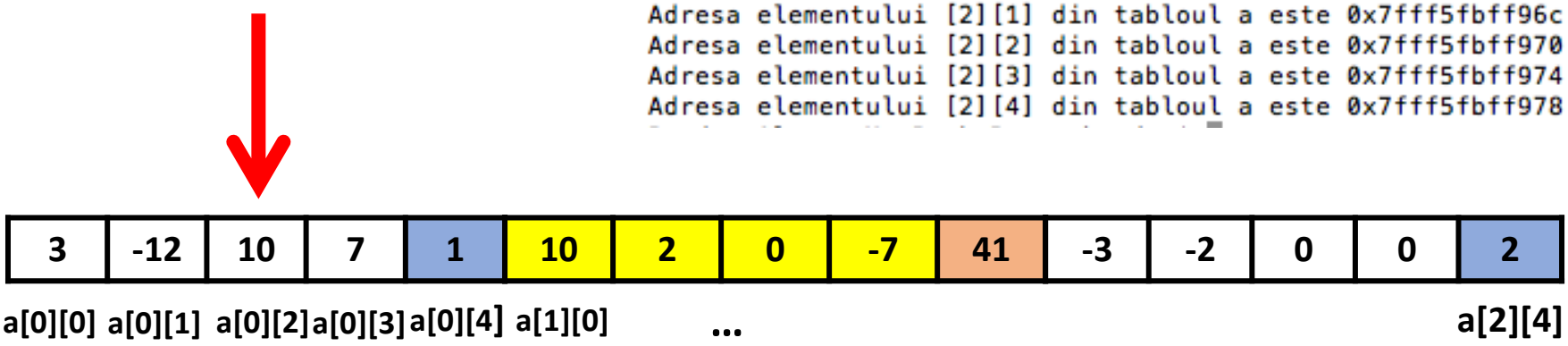


Tablouri bidimensionale

Definiție: set de valori de același tip memorat la adrese succesive de memorie.

	0	1	2	3	4
0	3	-12	10	7	1
1	10	2	0	-7	41
2	-3	-2	0	0	2

```
Adresa elementului [0][0] din tabloul a este 0x7fff5fbff940
Adresa elementului [0][1] din tabloul a este 0x7fff5fbff944
Adresa elementului [0][2] din tabloul a este 0x7fff5fbff948
Adresa elementului [0][3] din tabloul a este 0x7fff5fbff94c
Adresa elementului [0][4] din tabloul a este 0x7fff5fbff950
Adresa elementului [1][0] din tabloul a este 0x7fff5fbff954
Adresa elementului [1][1] din tabloul a este 0x7fff5fbff958
Adresa elementului [1][2] din tabloul a este 0x7fff5fbff95c
Adresa elementului [1][3] din tabloul a este 0x7fff5fbff960
Adresa elementului [1][4] din tabloul a este 0x7fff5fbff964
Adresa elementului [2][0] din tabloul a este 0x7fff5fbff968
Adresa elementului [2][1] din tabloul a este 0x7fff5fbff96c
Adresa elementului [2][2] din tabloul a este 0x7fff5fbff970
Adresa elementului [2][3] din tabloul a este 0x7fff5fbff974
Adresa elementului [2][4] din tabloul a este 0x7fff5fbff978
```



Reprezentarea în memoria calculatorului a unui tablou bidimensional

Tablouri bidimensionale

- **definiție: set de valori de același tip memorat la adrese succesive de memorie**
- **memorie:**
  - ✓ cantitatea de memorie necesară pentru stocarea unui tablou este direct proporțională cu tipul de date și mărimea sa.
  - ✓ tip nume [dimensiune1][dimensiune2] →

sizeof(nume) = sizeof (tip) * dimensiune1 * dimensiune2 ;

Tablouri bidimensionale

- cantitatea de memorie necesară pentru stocarea unui tablou este direct proporțională cu tipul de date și mărimea sa.
- tip nume [dimensiune1][dimensiune2]

sizeof(nume) = sizeof (tip) * dimensiune1 * dimensiune2 ;

Ce afișează programul?

```c
#include <stdio.h>

int main()
{
    int a[5][25];
    double v[100][50];
    char c[34][10];
    printf("Stocarea tabloului a necesita %d octeti. \n", sizeof(a));
    printf("Stocarea tabloului v necesita %d octeti. \n", sizeof(v));
    printf("Stocarea tabloului c necesita %d octeti. \n", sizeof(c));

    return 0;
}
```

Tablouri bidimensionale

- cantitatea de memorie necesară pentru stocarea unui tablou este direct proporțională cu tipul de date și mărimea sa.
- tip nume [dimensiune1][dimensiune2]

sizeof(nume) = sizeof (tip) * dimensiune1 * dimensiune2 ;

```
Stocarea tabloului a necesita 500 octeti.
Stocarea tabloului v necesita 40000 octeti.
Stocarea tabloului c necesita 340 octeti.
```

```c
#include <stdio.h>

int main()
{
    int a[5][25];
    double v[100][50];
    char c[34][10];
    printf("Stocarea tabloului a necesita %d octeti. \n", sizeof(a));
    printf("Stocarea tabloului v necesita %d octeti. \n", sizeof(v));
    printf("Stocarea tabloului c necesita %d octeti. \n", sizeof(c));

    return 0;
}
```

Tablouri bidimensionale

Citire și afișare:

```c
#include <stdio.h>

int main()
{
    int L, C, i, j;

    printf("Nr. de linii al matricei: "); scanf("%d", &L);
    printf("Nr. de coloane al matricei: "); scanf("%d", &C);
    int a[L][C];

    //citirea elementelor matricei a
    for(i = 0; i < L; i++)
        for(j = 0;j <C; j++)
        {
            printf("a[%d][%d] = ", i, j);
            scanf("%d", &a[i][j]);
        }
    //afisarea matricei a
    for(i = 0; i < L; i++)
    {
        for(j = 0;j <C; j++)
            printf("%d\t", a[i][j]);
        printf("\n");
    }

    return 0;
}
```

valabil în standardul C99 (vezi declarațiile de tablouri în exemplele anterioare)

Tablouri bidimensionale

Citire și afișare:

```c
#include <stdio.h>

int main()
{
    int L, C, i, j;

    printf("Nr. de linii al matricei: "); scanf("%d", &L);
    printf("Nr. de coloane al matricei: "); scanf("%d", &C);
    int a[L][C];

    //citirea elementelor matricei a
    for(i = 0; i < L; i++)
        for(j = 0;j <C; j++)
        {
            printf("a[%d][%d] = ", i, j);
            scanf("%d", &a[i][j]);
        }
    //afisarea matricei a
    for(i = 0; i < L; i++)
    {
        for(j = 0;j <C; j++)
            printf("%d\t", a[i][j]);
        printf("\n");
    }

    return 0;
}
```

```
Nr. de linii al matricei: 2
Nr. de coloane al matricei: 3
a[0][0] = 1
a[0][1] = 3
a[0][2] = 6
a[1][0] = 4
a[1][1] = 8
a[1][2] = 0
1       3       6
4       8       0
```

Cursul de azi

1. Pointeri
2. Tablouri
3. **Șiruri de caractere**
4. **Structuri**

Șiruri de caractere

- **un șir de caractere (string)** este o zonă de memorie ocupată cu caractere/char-uri (un char ocupă un octet) terminată cu un octet de valoare 0 (caracterul '\0' are codul ASCII egal cu 0).

- o variabilă care reprezintă un șir de caractere este un pointer (adresa) la primul octet.

- se poate reprezenta ca:
  - **tablou de caractere (pointer constant):**

```
char sir1[10];
char sir2[10] = ”exemplu”;
```

  - **pointer la caractere:**

```
char *sir3;
char *sir4 = ”exemplu”;
```

Codurile ASCII

cod ASCII = reprezentarea numerică a unui caracter
(ASCII – American Standard Code for Information Interchange)

```c
#include <stdio.h>

int main()
{
    int i;

    for(i = 33; i <= 127; i++)
    {
        printf("%c ", i);
        if((i - 2) % 10 == 0)
            printf("\n");
    }

    return 0;
}
```

Ce afișează programul?

Codurile ASCII

cod ASCII = reprezentarea numerică a unui caracter
(ASCII – American Standard Code for Information Interchange)

```c
#include <stdio.h>

int main()
{
    int i;

    for(i = 33; i <= 127; i++)
    {
        printf("%c ", i);
        if((i - 2) % 10 == 0)
            printf("\n");
    }

    return 0;
}
```

```
! " # $ % & ' ( ) *
+ , - . / 0 1 2 3 4
5 6 7 8 9 : ; < = >
? @ A B C D E F G H
I J K L M N O P Q R
S T U V W X Y Z [ \
] ^ _ ` a b c d e f
g h i j k l m n o p
q r s t u v w x y z
{ | } ~
```

Codurile ASCII

Dec	Hx	Oct	Char	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr
0	0	000	NUL (null)	32	20	040		Space	64	40	100	@	@	96	60	140	`	`
1	1	001	SOH (start of heading)	33	21	041	!	!	65	41	101	A	A	97	61	141	a	a
2	2	002	STX (start of text)	34	22	042	"	"	66	42	102	B	B	98	62	142	b	b
3	3	003	ETX (end of text)	35	23	043	#	#	67	43	103	C	C	99	63	143	c	c
4	4	004	EOT (end of transmission)	36	24	044	$	$	68	44	104	D	D	100	64	144	d	d
5	5	005	ENQ (enquiry)	37	25	045	%	%	69	45	105	E	E	101	65	145	e	e
6	6	006	ACK (acknowledge)	38	26	046	&	&	70	46	106	F	F	102	66	146	f	f
7	7	007	BEL (bell)	39	27	047	'	'	71	47	107	G	G	103	67	147	g	g
8	8	010	BS (backspace)	40	28	050	(	(	72	48	110	H	H	104	68	150	h	h
9	9	011	TAB (horizontal tab)	41	29	051	)	)	73	49	111	I	I	105	69	151	i	i
10	A	012	LF (NL line feed, new line)	42	2A	052	*	*	74	4A	112	J	J	106	6A	152	j	j
11	B	013	VT (vertical tab)	43	2B	053	+	+	75	4B	113	K	K	107	6B	153	k	k
12	C	014	FF (NP form feed, new page)	44	2C	054	,	,	76	4C	114	L	L	108	6C	154	l	l
13	D	015	CR (carriage return)	45	2D	055	-	-	77	4D	115	M	M	109	6D	155	m	m
14	E	016	SO (shift out)	46	2E	056	.	.	78	4E	116	N	N	110	6E	156	n	n
15	F	017	SI (shift in)	47	2F	057	/	/	79	4F	117	O	O	111	6F	157	o	o
16	10	020	DLE (data link escape)	48	30	060	0	0	80	50	120	P	P	112	70	160	p	p
17	11	021	DC1 (device control 1)	49	31	061	1	1	81	51	121	Q	Q	113	71	161	q	q
18	12	022	DC2 (device control 2)	50	32	062	2	2	82	52	122	R	R	114	72	162	r	r
19	13	023	DC3 (device control 3)	51	33	063	3	3	83	53	123	S	S	115	73	163	s	s
20	14	024	DC4 (device control 4)	52	34	064	4	4	84	54	124	T	T	116	74	164	t	t
21	15	025	NAK (negative acknowledge)	53	35	065	5	5	85	55	125	U	U	117	75	165	u	u
22	16	026	SYN (synchronous idle)	54	36	066	6	6	86	56	126	V	V	118	76	166	v	v
23	17	027	ETB (end of trans. block)	55	37	067	7	7	87	57	127	W	W	119	77	167	w	w
24	18	030	CAN (cancel)	56	38	070	8	8	88	58	130	X	X	120	78	170	x	x
25	19	031	EM (end of medium)	57	39	071	9	9	89	59	131	Y	Y	121	79	171	y	y
26	1A	032	SUB (substitute)	58	3A	072	:	:	90	5A	132	Z	Z	122	7A	172	z	z
27	1B	033	ESC (escape)	59	3B	073	;	;	91	5B	133	[	[	123	7B	173	{	{
28	1C	034	FS (file separator)	60	3C	074	<	<	92	5C	134	\	\	124	7C	174			|
29	1D	035	GS (group separator)	61	3D	075	=	=	93	5D	135	]	]	125	7D	175	}	}
30	1E	036	RS (record separator)	62	3E	076	>	>	94	5E	136	^	^	126	7E	176	~	~
31	1F	037	US (unit separator)	63	3F	077	?	?	95	5F	137	_	_	127	7F	177	&#127;	DEL

Codurile ASCII

Extended ASCII Codes

128	Ç	144	É	160	á	176	▒	192	└	208	╨	224	α	240	≡
129	ü	145	æ	161	í	177	▒	193	┴	209	╤	225	ß	241	±
130	é	146	Æ	162	ó	178	▓	194	┬	210	╥	226	Γ	242	≥
131	â	147	ô	163	ú	179	│	195	├	211	╙	227	π	243	≤
132	ä	148	ö	164	ñ	180	┤	196	─	212	╘	228	Σ	244	⌠
133	à	149	ò	165	Ñ	181	╡	197	┼	213	╒	229	σ	245	⌡
134	å	150	û	166	ª	182	╢	198	╞	214	╓	230	µ	246	÷
135	ç	151	ù	167	º	183	╖	199	╟	215	╫	231	τ	247	≈
136	ê	152	ÿ	168	¿	184	╕	200	╚	216	╪	232	Φ	248	°
137	ë	153	Ö	169	⌐	185	╣	201	╔	217	┘	233	Θ	249	·
138	è	154	Ü	170	¬	186	║	202	╩	218	┌	234	Ω	250	·
139	ï	155	¢	171	½	187	╗	203	╦	219	█	235	δ	251	√
140	î	156	£	172	¼	188	╝	204	╠	220	▄	236	∞	252	n
141	ì	157	¥	173	¡	189	╜	205	═	221	▌	237	φ	253	²
142	Ä	158	₧	174	«	190	╛	206	╬	222	▐	238	ε	254	■
143	Å	159	ƒ	175	»	191	┐	207	╧	223	▀	239	∩	255	

www.LookupTables.com

Citirea și afișarea șirurilor de caractere

- **citire:**
  - ✓ funcția **scanf** cu **modelatorul de format %s**:
    Atenție: dacă inputul este un șir de caractere cu spațiu, citește până la spațiu
  - ✓ funcția **fgets/ gets** (revenim în cursurile următoare)
    - citește și spațiile

- **afișare:**
  - ✓ funcția **printf** cu **modelatorul de format %s**:
  - ✓ funcția **puts**
    - trece pe linia următoare

https://www.geeksforgeeks.org/fgets-gets-c-language/

Citirea și afișarea șirurilor de caractere

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    char sir1[100], sir2[100];
    printf("Dati sirul 1: ");
    fgets(sir1,100, stdin);
    printf("S-a citit sirul: ");
    puts(sir1);

    printf("Dati un al 2-lea sir: ");
    scanf("%s", sir2);
    printf("S-a citit sirul: %s \n", sir2);

    return 0;
}
```

Citirea și afișarea șirurilor de caractere

```
Dati sirul 1: Avem curs de PC.
S-a citit sirul: Avem curs de PC.

Dati un al 2-lea sir: Azi nu avem seminar.
S-a citit sirul: Azi
```

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    char sir1[100], sir2[100];
    printf("Dati sirul 1: ");
    fgets(sir1,100, stdin);
    printf("S-a citit sirul: ");
    puts(sir1);

    printf("Dati un al 2-lea sir: ");
    scanf("%s", sir2);
    printf("S-a citit sirul: %s \n", sir2);

    return 0;
}
```

Programa cursului

1. Introducere

- Algoritmi
- Limbaje de programare.
- Introducere în limbajul C. Structura unui program C.

2. Fundamentele limbajului C

- Etapele realizării unui program C.
- Tipuri de date fundamentale. Variabile. Constante. Operatori. Expresii. Conversii.
- Tipuri derivate de date: tablouri, șiruri de caractere, structuri, uniuni, câmpuri de biți, enumerări, pointeri
- Instrucțiuni de control
- Directive de preprocesare. Macrodefiniții.
- Funcții de citire/scriere.

3. Fișiere text

- Funcții specifice de manipulare.

4. Funcții (1)

- Declarare și definire. Apel. Metode de trasmitere a paramerilor. Pointeri la funcții.

5. Tablouri și pointeri

- Legătura dintre tablouri și pointeri
- Aritmetica pointerilor
- Alocarea dinamică a memoriei
- Clase de memorare

6. Șiruri de caractere

- Funcții specifice de manipulare

7. Fișiere binare

- Funcții specifice de manipulare

8. Structuri de date complexe și autoreferite

- Definire și utilizare

9. Funcții (2)

- Funcții cu număr variabil de argumente
- Preluarea argumentelor funcției main din linia de comandă
- Programare generică

Cursul de azi

1. Pointeri
2. Tablouri
3. Șiruri de caractere
4. **Structuri**

Tipuri de date structurate

Limbajul C permite crearea tipurilor de date în 5 moduri:

a. **structura/ articol/ înregistrare (struct)** – grupează mai multe variabile sub același nume;
b. **câmpul de biți (variațiune a structurii)** → acces ușor la biții individuali
c. **uniunea (union)** – face posibil ca aceleași zone de memorie să fie definite ca două sau mai multe tipuri diferite de variabile
d. **enumerarea (enum)** – listă de constante întregi cu nume
e. **tip definit de utilizator (typedef)** – definește un nou nume pentru un tip existent

a. Structuri

- variabile grupate sub același nume
- **sintaxa:**

struct <nume> {

< tip 1 > <variabila 1>;

< tip 2 > <variabila 2>;

< tip n > <variabila n>;

} lista_identificatori_de_tip_struct;

- variabilele care fac parte din structură sunt denumite **membri** (**elemente** sau **câmpuri**) ai structurii.

Structuri

```
struct <nume> { < tip 1 >      <variabila 1>;
                < tip 2 >      <variabila 2>;
                ------------------------------
                < tip n >      <variabila n>; } lista_identificatori;
```

Obs. 1:

- ✓dacă numele structurii (<nume>) lipsește, structura se numește **anonimă**.
- ✓Dacă lista identificatorilor declarați lipsește, se definește doar tipul structură.
- ✓Cel puțin una dintre aceste specificații trebuie să existe.

Obs. 2: dacă <nume> este prezent → se pot declara noi variabile de tip structura:

struct <nume> <lista noilor identificatori>;

Obs. 3: referirea unui membru al unei variabile de tip structură → **operatorul de selecție punct .** care precizează identificatorul variabilei și al câmpului.

```c
#include <stdio.h>
struct Data
    {int zi, luna, an;};
Data azi={27,10,2024};
int main()
{
    printf("azi este %d.%d.%d \n", azi.zi, azi.luna, azi.an);
    return 0;
}
```

```
ERROR!
/tmp/9z8aqffi14.c:4:1: error: unknown type name 'Data'; use 'struct' keyword to refer to
    the type
    4 | Data azi={27,10,2024};
      | ^~~~
      | struct
/tmp/9z8aqffi14.c:4:14: warning: excess elements in scalar initializer
    4 | Data azi={27,10,2024};
```

```c
#include <stdio.h>
struct Data
    {int zi, luna, an;};
struct Data azi={27,10,2024};
int main()
{
    printf("azi este %d.%d.%d \n", azi.zi, azi.luna, azi.an);
    return 0;
}
```

```
azi este 27.10.2024


=== Code Execution Successful ===
```

```cpp
#include <iostream>
using namespace std;

struct Data
    {int zi, luna, an;};
Data azi={27,10,2024};
int main()
{
    cout<< azi.zi<<'.'<< azi.luna<<'.'<< azi.an;
    return 0;
}
```

```
27.10.2024

=== Code Execution Successful ===
```

```cpp
#include <stdio.h>
struct Data
    {int zi, luna, an;};
struct Data azi={27,10,2024};
int main()
{
    printf("azi este %d.%d.%d \n", azi.zi, azi.luna, azi.an);
    return 0;
}
```

```
azi este 27.10.2024

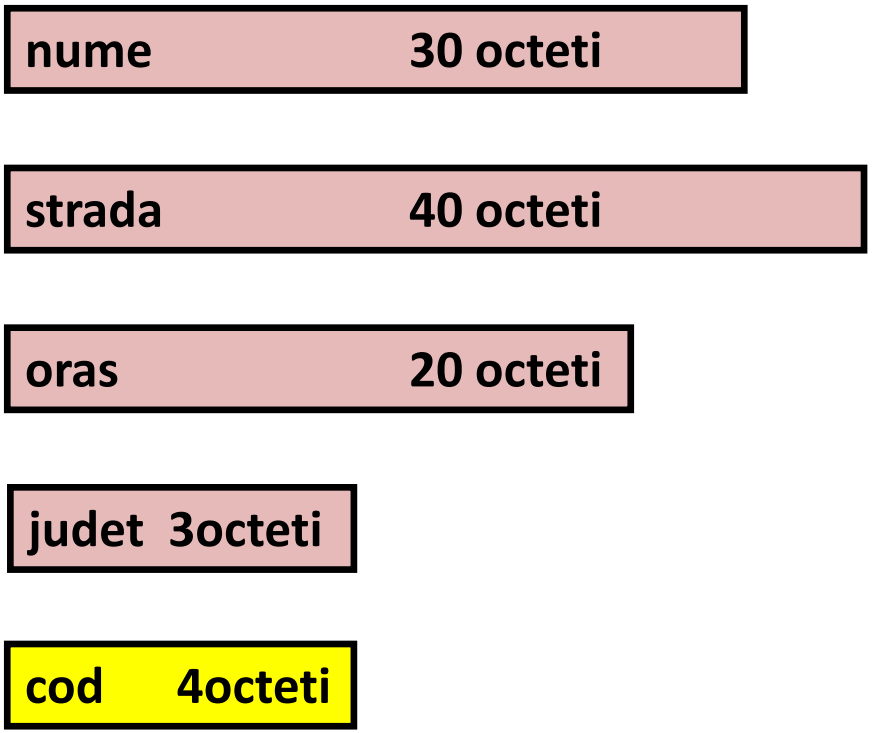

=== Code Execution Successful ===
```

Structuri. Exemplu

```
struct adrese {
        char nume[30];
        char strada[40];
        char oras[20], judet[3];
        int cod;};
struct adrese A;
```

- numele ***adrese*** identifică această structură de date particulară
- **struct adrese A;** declară o variabilă de tip adrese și îi alocă memorie

Structura din memorie pentru variabila A de tip adrese



Structuri. Exemplu

```c
#include <stdio.h>

int main()
{
    struct adrese {
    char nume[30];
    char strada[40];
    char oras[20], judet[3];
    int cod;
    };
    struct adrese A;

    printf("%d\t", (int)sizeof(A.nume));
    printf("%d\t", (int)sizeof(A.cod));
    printf("%d", (int)sizeof(A));

    return 0;
}
```

Ce afișează programul?

30 **4** **100**

Compilatorul alocă memorie în plus pentru aliniere (multiplu de 4)

Structuri. Exemplu

```
1  #include <stdio.h>
2
3  int main()
4  {
5      struct adrese {
6      char nume[30];
7      char strada[40];
8      char oras[20], judet[6];
9      int cod;
10     };
11     struct adrese A;
12
13     printf("%d\t", (int)sizeof(A.nume));
14     printf("%d\t", (int)sizeof(A.cod));
15     printf("%d", (int)sizeof(A));
16
17     return 0;
18 }
```

Ce afișează programul?

30 4 100

Structuri. Exemplu

```
1  #include <stdio.h>
2
3  int main()
4  {
5      struct adrese {
6      char nume[30];
7      char strada[40];
8      char oras[20], judet[3];
9      };
10     struct adrese A;
11
12     printf("%d\t", (int)sizeof(A.nume));
13     printf("%d\t", (int)sizeof(A));
14
15     return 0;
16 }
```

Ce afișează programul?

30 **93**

Compilatorul alocă memorie în plus pentru aliniere numai când avem tipuri de date diferite.

Structuri. Exemplu

```
struct adrese {
        char nume[30];
        char strada[40];
        char oras[20];
        char judet[3];
        int cod;
        } A, B, C;
```

- Definește un tip de structură numit ***adrese***
- Declară ca fiind de acest tip variabilele **A, B, C**

```
struct {
        char nume[30];
        char strada[40];
        char oras[20];
        char judet[3];
        int cod;
        } A;
```

- Declară o variabilă numită **A** definită de structura care o precede.

Structuri. Exemplu

```
struct adrese {
        char nume[30];
        char strada[40];
        char oras[20];
        char judet[3];
        int cod;
        } A, B, C;
```

- Definește un tip de structură numit ***adrese***
- Declară ca fiind de acest tip variabilele **A, B, C**

- **accesul la membrii structurii** se face prin folosirea operatorului punct:

  nume_variabila.nume_camp

- **citirea:**

  scanf("%d", &C.cod);

- **atribuiri pentru variabile de tip structură:**

  B = A;

Obs. Singura operație cu structuri este atribuirea.

Cursul 3. Tipuri derivate de date (1):

1. **Pointeri**
2. **Tablouri**
3. **Șiruri de caractere**
4. **Structuri**

Cursul 4.

1. Tipuri derivate de date (2):
   uniuni, câmpuri de biți, enumerări
2. Instrucţiuni de control